49. Design GitHub Copilot

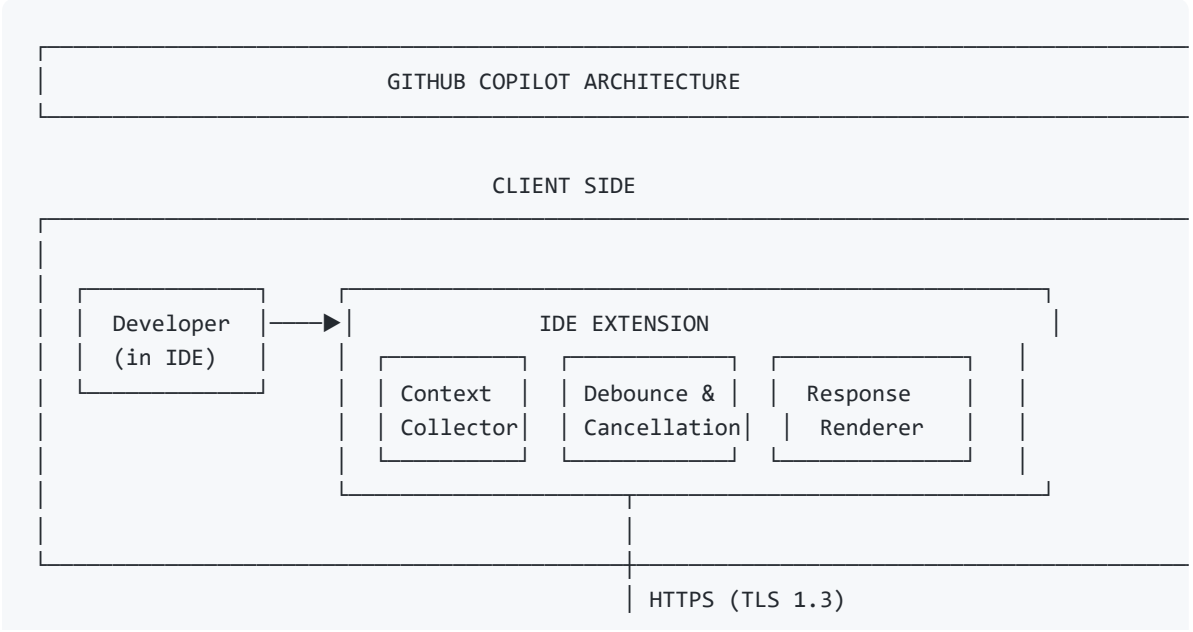
Problem Statement

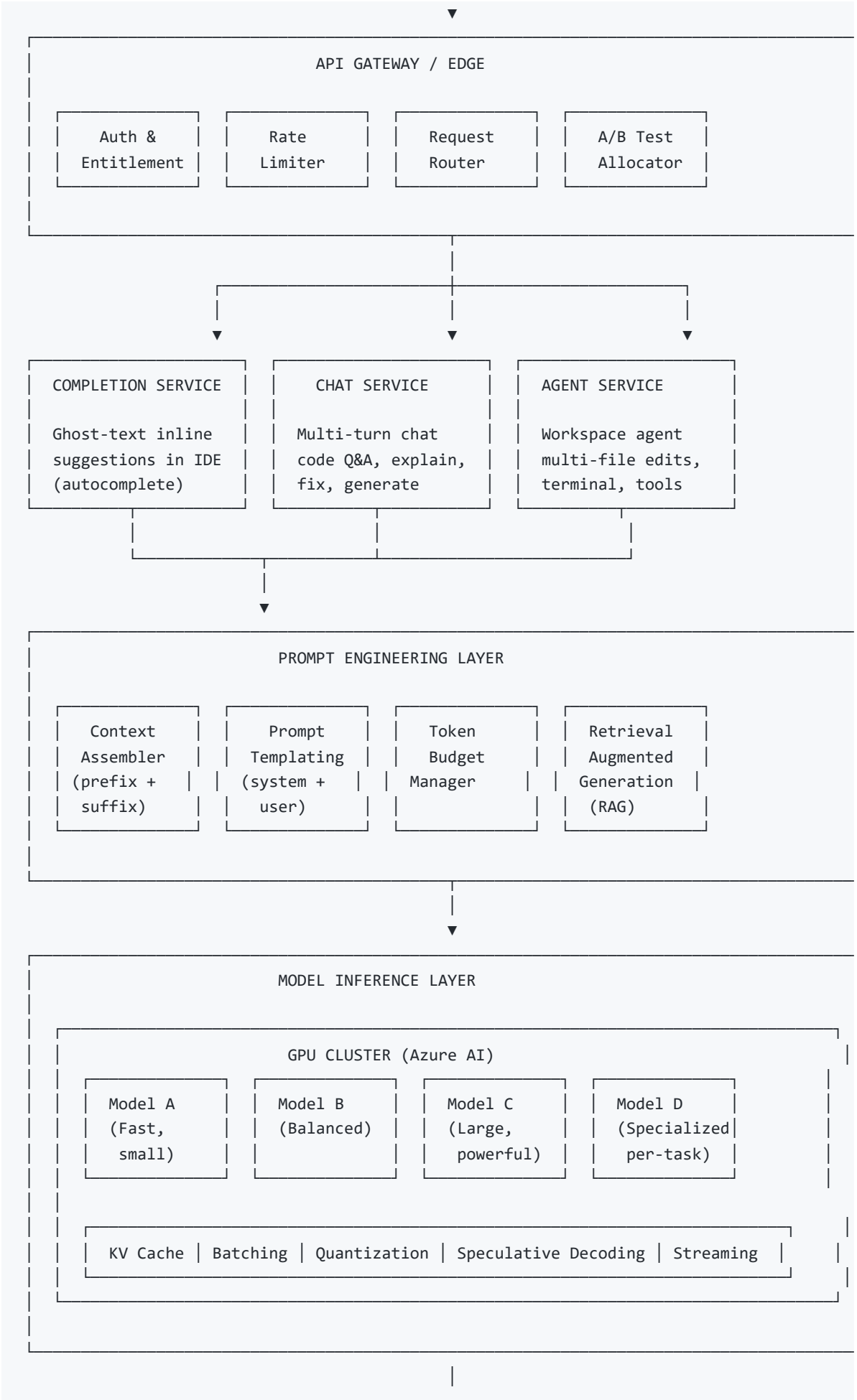
Design an AI-powered code completion and assistance platform like GitHub Copilot. The system should provide real-time code suggestions, multi-line completions, chat-based code generation, and contextual understanding of codebases — all integrated into IDEs at massive scale.

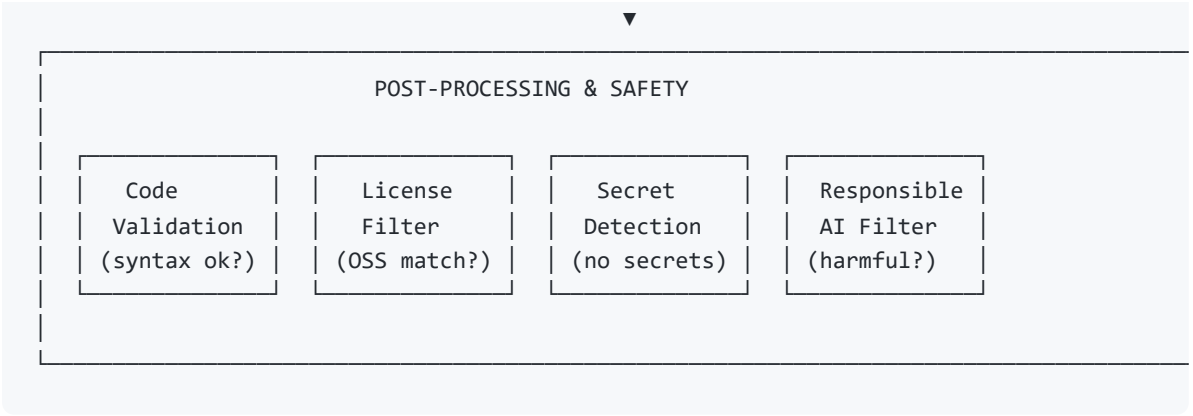
Clarifying Questions to Ask

1. **Latency**: What's the acceptable latency for inline suggestions? (< 500ms typical)
2. **Context window**: How much surrounding code context do we send to the model?
3. **Model hosting**: Self-hosted models or third-party LLM API (OpenAI)?
4. **Multi-modal**: Code completions only, or also chat, PR summaries, CLI assistance?
5. **Languages**: How many programming languages to support? (30+ typical)
6. **Privacy**: Can we use customer code for model training? Telemetry policies?
7. **Offline**: Should suggestions work offline or always require connectivity?
8. **Rate limiting**: Per-user or per-organization rate limits?

High-Level Design




API GATEWAY / EDGE
Auth & Entitlement
Rate Limiter
Request Router
A/B Test Allocator
COMPLETION SERVICE
Ghost-text inline suggestions in IDE (autocomplete)
CHAT SERVICE
Multi-turn chat code Q&A, explain, fix, generate
AGENT SERVICE
Workspace agent multi-file edits, terminal, tools
PROMPT ENGINEERING LAYER
Context Assembler (prefix + suffix)
Prompt Templating (system + user)
Token Budget Manager
Retrieval Augmented Generation (RAG)
MODEL INFERENCE LAYER
GPU CLUSTER (Azure AI)
Model A (Fast, small)
Model B (Balanced)
Model C (Large, powerful)
Model D (Specialized per-task)
KV Cache | Batching | Quantization | Speculative Decoding | Streaming



API Design

Completion API (Inline Suggestions)

Method	Endpoint	Description
`POST`	`/v1/completions`	Get inline code completion
`POST`	`/v1/completions/accept`	Log acceptance telemetry
`POST`	`/v1/completions/reject`	Log rejection telemetry

Chat API

Method	Endpoint	Description
`POST`	`/v1/chat`	Send chat message (streaming SSE)
`GET`	`/v1/chat/{sessionId}/history`	Retrieve chat history
`DELETE`	`/v1/chat/{sessionId}`	Delete chat session

Agent API

Method	Endpoint	Description
`POST`	`/v1/agents/run`	Start an agent task
`GET`	`/v1/agents/{runId}/status`	Poll agent run status
`POST`	`/v1/agents/{runId}/feedback`	Submit tool-call approval/rejection

Completion Request

```
POST /v1/completions
{
  "prompt": {
    "prefix": "def fibonacci(n):\n    if n <= 1:\n        return n\n    ",
    "suffix": "\n\nprint(fibonacci(10))",
    "language": "python",
    "file_path": "math_utils.py"
  },
  "context": {
    "open_files": [
      {
        "path": "tests/test_math.py",
        "content": "...(relevant snippet)...",
        "relevance_score": 0.87
      }
    ],
    "repo_info": {
      "languages": ["python"],
      "framework": "pytest"
    }
  },
  "params": {
    "max_tokens": 256,
    "temperature": 0.0,
    "top_p": 1.0,
    "n": 3,
    "stop": ["\ndef ", "\nclass ", "\n\n"]
  },
  "user": {
    "ide": "vscode",
    "ide_version": "1.95.0",
    "extension_version": "1.200.0"
  }
}
```

Completion Response

```
{
  "id": "cmpl_abc123",
  "choices": [
    {
      "index": 0,
      "text": "return fibonacci(n-1) + fibonacci(n-2)",
      "finish_reason": "stop",
      "confidence": 0.94
    }
  ],
  "usage": {
    "prompt_tokens": 312,
    "completion_tokens": 14
  },
```

```
  "telemetry_id": "tel_xyz789"
}
```

Chat Request (Streaming)

```
POST /v1/chat
{
  "session_id": "sess_001",
  "messages": [
    { "role": "system", "content": "You are a helpful coding assistant..." },
    { "role": "user", "content": "Explain this function and suggest improvements" }
  ],
  "context": {
    "active_selection": "def fib(n): return n if n<=1 else fib(n-1)+fib(n-2)",
    "file_path": "math_utils.py",
    "language": "python"
  },
  "stream": true
}
```

Context Collection & Prompt Assembly

The most critical differentiator — how we build the prompt from surrounding code.

Context Sources (Priority Order)

```
┌──────────────────────────────────────────────────────────────┐
│                  CONTEXT ASSEMBLY PIPELINE                   │
│                                                              │
│  Priority 1: IMMEDIATE CONTEXT                               │
│  ├── Current file prefix (code before cursor)                │
│  ├── Current file suffix (code after cursor)                 │
│  └── Cursor position, indentation level                      │
│                                                              │
│  Priority 2: NEIGHBOR CONTEXT                                │
│  ├── Open editor tabs (ranked by recency)                    │
│  ├── Imports/includes in current file → resolve to definitions │
│  └── Symbols referenced near cursor → find declarations      │
│                                                              │
│  Priority 3: REPOSITORY CONTEXT (RAG)                        │
│  ├── Embedding search over repo index                        │
│  ├── Similar code patterns in the codebase                   │
│  └── README, docs, config files                              │
│                                                              │
│  Priority 4: LEARNED PATTERNS                                │
│  ├── User's accept/reject history                            │
```

```
│  ├── Organization coding standards                                  │
│  └── Custom instructions (.github/copilot-instructions.md)         │
│                                                                     │
│  ──────────────────────────────────────────                         │
│  Token Budget: ~8K context → model                                  │
│  Strategy: Fill-in-the-Middle (FIM) encoding                        │
│  Format: <PRE> prefix <SUF> suffix <MID> → completion               │
│                                                                     │
└─────────────────────────────────────────────────────────────────────┘
```

Fill-in-the-Middle (FIM) Encoding

```
Standard autocompletion (left-to-right):
  prompt = prefix → generate next tokens

FIM (Copilot uses this):
  prompt = <PRE> {prefix} <SUF> {suffix} <MID>
  model generates tokens to fill the gap

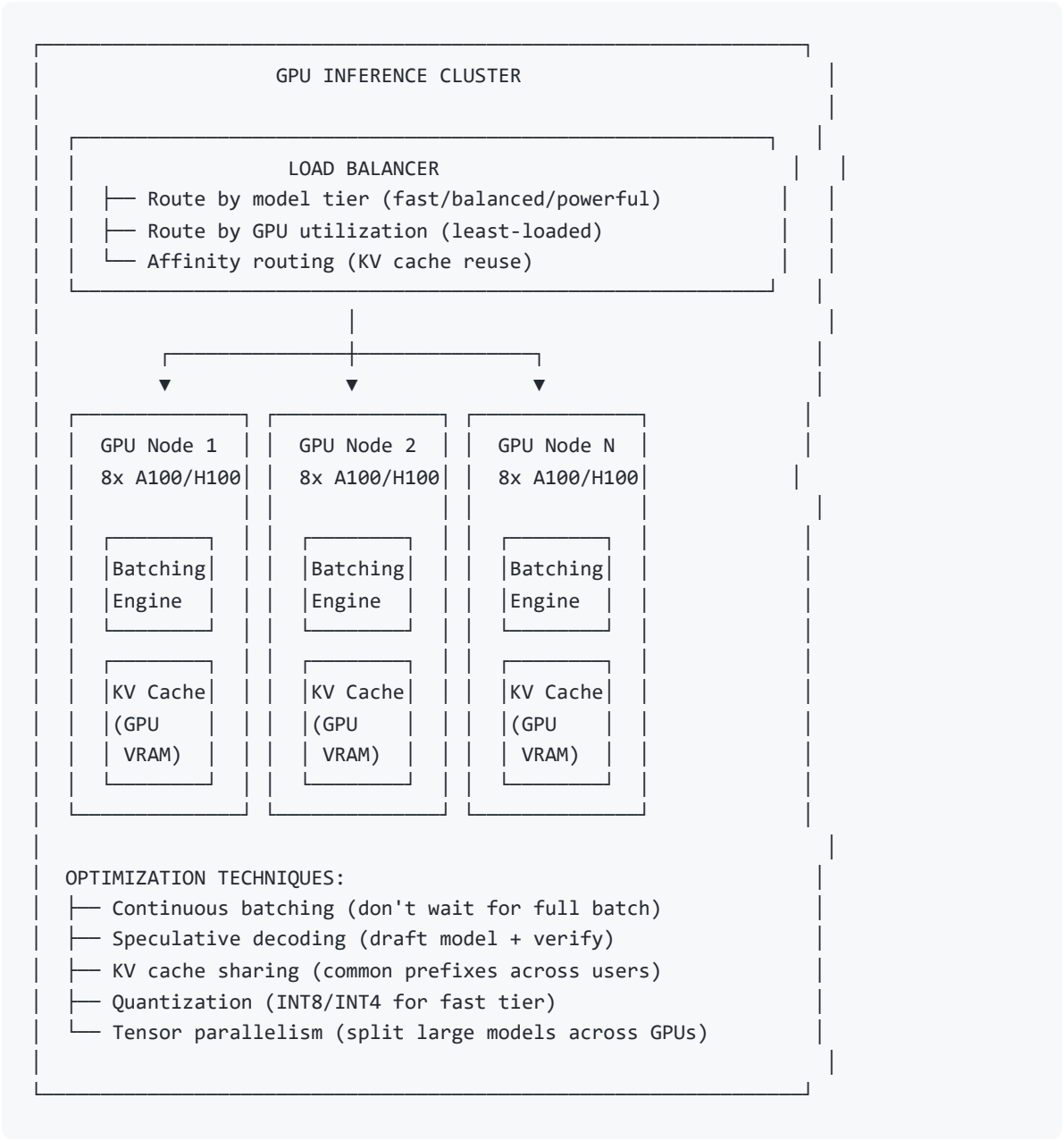
Why FIM is better:
  - Sees code AFTER the cursor (function signature, tests, etc.)
  - Produces completions that fit surrounding code
  - Better for mid-function edits, not just end-of-file
```

Model Inference Layer

Multi-Model Strategy

Model Tier	Use Case	Latency Target	Example
Fast	Inline completions, single-line	< 300ms	Small distilled model (1-7B params)
Balanced	Multi-line completions, chat	< 1s	Medium model (13-30B params)
Powerful	Complex generation, agent tasks	< 10s	Large model (70B+ / GPT-4 class)
Specialized	Code review, security scanning	Async	Fine-tuned per-task models

GPU Cluster Architecture



Latency Breakdown (Inline Completion)

```
End-to-end target: < 500ms

IDE Extension:
  ├── Debounce wait                    ~150ms (wait for typing pause)
  ├── Context collection                ~20ms
  └── Serialize & send request          ~5ms

Network:
  └── Round-trip to nearest region      ~30ms

API Gateway:
  ├── Auth + entitlement check          ~5ms
  ├── Rate limit check                  ~2ms
  └── Route to inference cluster        ~3ms
```

```
Prompt Engineering:
  ├── Context assembly (FIM encoding)    ~10ms
  ├── RAG retrieval (if enabled)         ~30ms
  └── Token counting & truncation        ~5ms

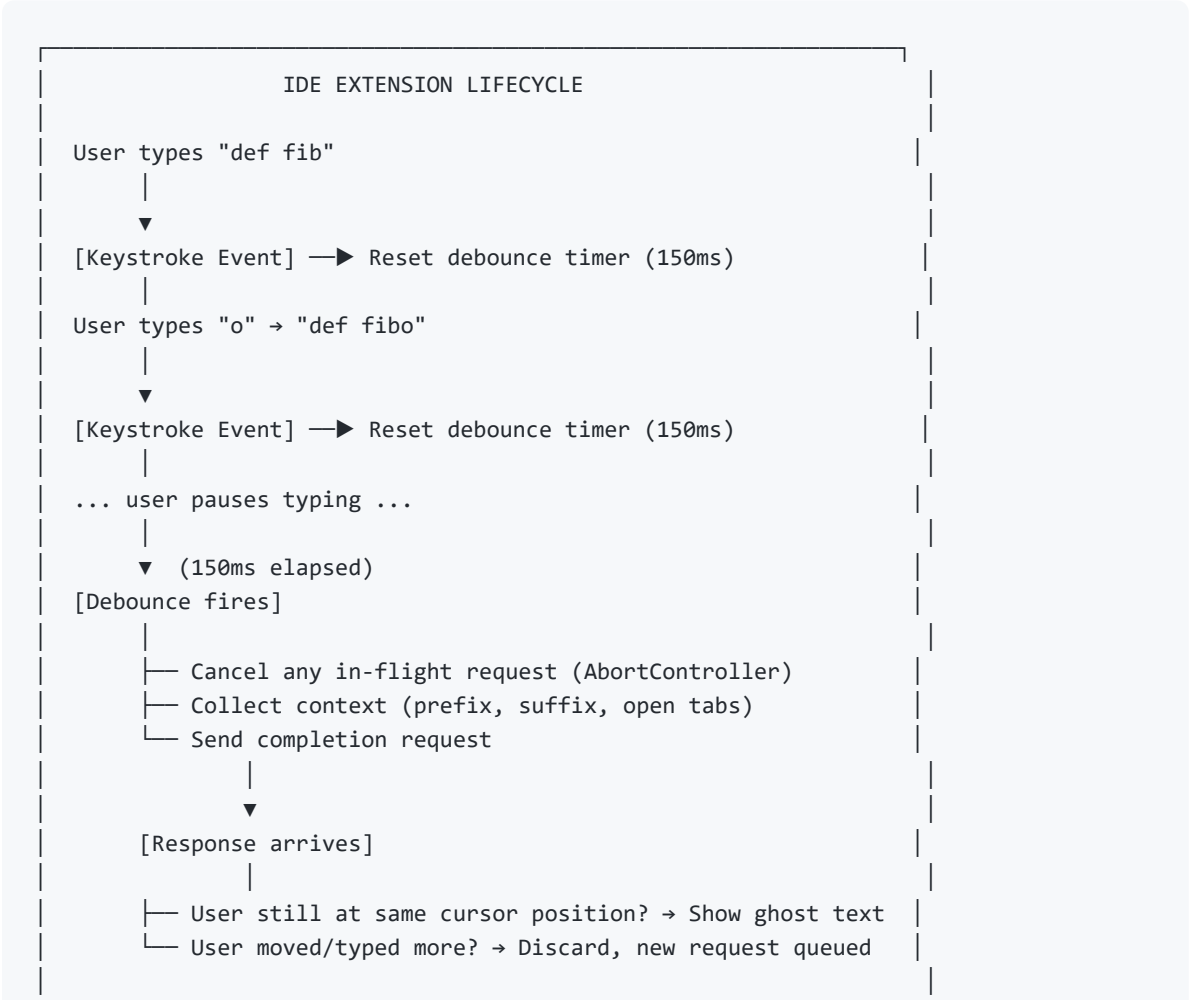
Model Inference:
  ├── Queuing (wait for batch slot)      ~20ms
  ├── Prefill (process prompt tokens)    ~30ms
  └── Decode (generate ~30 tokens)       ~80ms

Post-processing:
  ├── Syntax validation                  ~5ms
  ├── License filter                     ~10ms
  └── Secret detection                   ~5ms

Total server-side:                      ~235ms
Total with network:                     ~265ms
Total with debounce:                    ~415ms
```

Debounce, Cancellation & Streaming

Client-Side Intelligence



```
│  CANCELLATION SCENARIOS:                                                │
│  ├── User types → cancel pending request                                │
│  ├── User moves cursor → cancel pending request                         │
│  ├── User switches file → cancel pending request                        │
│  └── User accepts/rejects → log telemetry                               │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

Why Cancellation Matters

```
Without cancellation:
  - 100M developers × 20 requests/min = 2B requests/min
  - GPU cost: astronomical

With cancellation:
  - 70-80% of requests cancelled before completion
  - Only ~20-30% actually reach inference
  - Massive cost savings
```

Post-Processing & Safety

License Filter (Duplicate Code Detection)

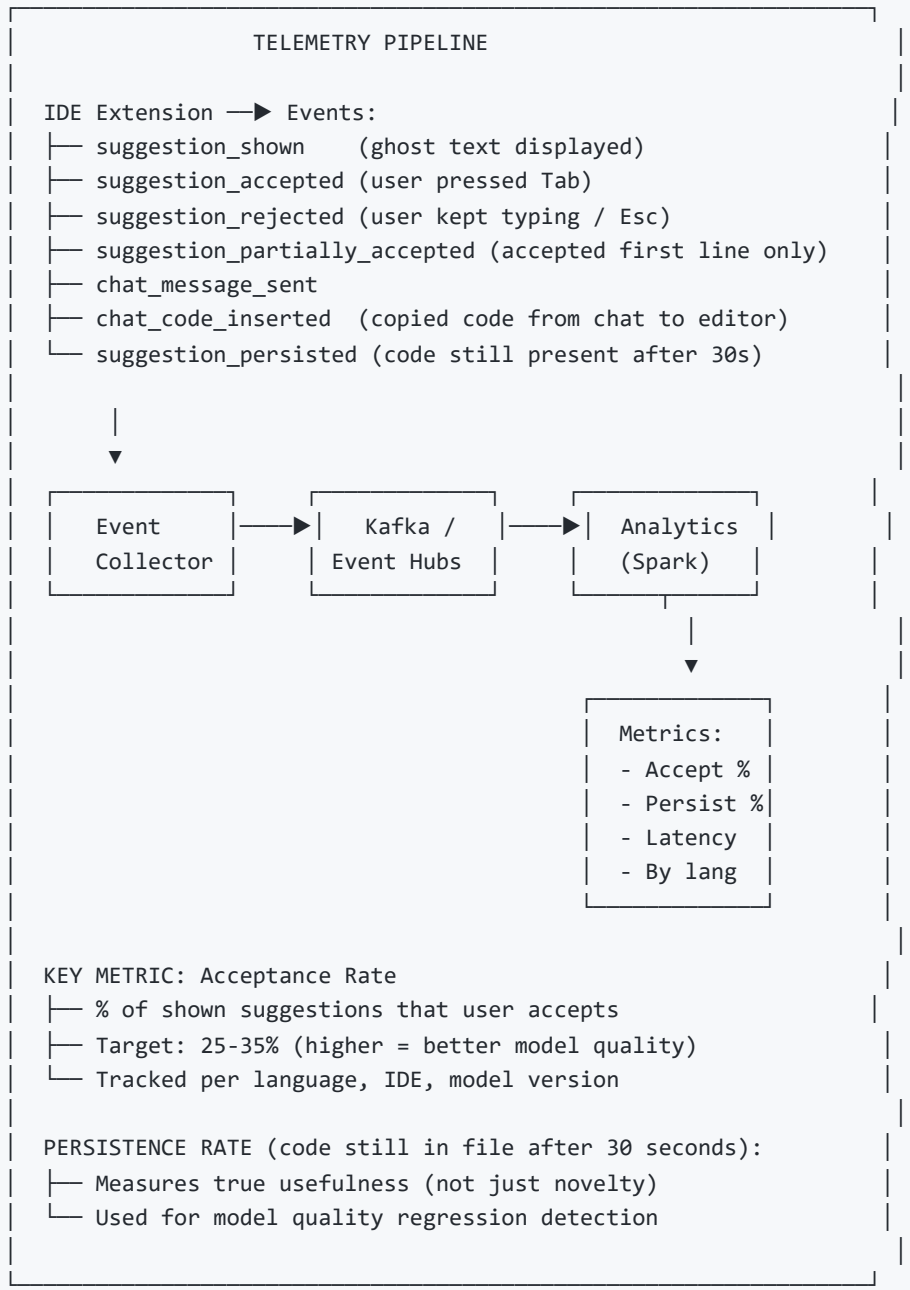
```
┌──────────────────────────────────────────────────────────────────────────┐
│                  LICENSE FILTER PIPELINE                                 │
│                                                                          │
│  Model output: "function sort(arr) { ... 30 lines ... }"                 │
│       │                                                                  │
│       ▼                                                                  │
│  1. FINGERPRINT: Normalize whitespace, compute n-gram hashes             │
│       │                                                                  │
│       ▼                                                                  │
│  2. SEARCH: Compare against known open-source code index                 │
│     (Bloom filter for fast pre-check, then exact match)                  │
│       │                                                                  │
│       ▼                                                                  │
│  3. MATCH FOUND?                                                         │
│     ├── No match → Pass through (safe to suggest)                        │
│     ├── Permissive license (MIT, Apache) → Allow, cite source            │
│     └── Copyleft license (GPL) → Block suggestion                        │
│                                                                          │
│  INDEX: ~hundreds of millions of public code files                        │
│  Storage: Bloom filter (~2GB RAM) + search index (distributed)           │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

Secret Detection

```
Scan model output for:
├── API keys (pattern: /[A-Za-z0-9]{32,}/)
├── Connection strings (pattern: /Server=.*;Password=.*/)
├── Private keys (pattern: /-----BEGIN.*PRIVATE KEY-----/)
├── Tokens (pattern: /ghp_[A-Za-z0-9]{36}/)
└── Custom patterns per organization

Action: Redact or block suggestion if secrets detected
```

Telemetry & Feedback Loop



Key Components

Component	Technology	Purpose
IDE Extension	VS Code API, LSP	Collect context, render suggestions, handle accept/reject
API Gateway	Azure API Management / Envoy	Auth, rate limiting, routing, TLS termination
Completion Service	Go / Rust microservice	Orchestrate prompt assembly → inference → post-processing
Chat Service	Go / Rust microservice	Multi-turn conversation management, streaming SSE
Agent Service	Go / Python microservice	Tool orchestration, multi-step task execution
Prompt Engine	Python / Rust	FIM encoding, context ranking, token budget management
RAG Index	Vector DB (pgvector / Qdrant)	Semantic code search over repository embeddings
Model Serving	vLLM / TensorRT-LLM / Triton	GPU inference with continuous batching, KV caching
GPU Cluster	Azure ND-series (A100/H100)	Tensor parallel model hosting, auto-scaling
License Filter	Bloom filter + search index	Detect code matching open-source with restrictive licenses
Secret Scanner	Regex + ML classifier	Block suggestions containing secrets or credentials
Telemetry	Kafka / Azure Event Hubs + Spark	Track acceptance rate, latency, usage metrics
Feature Flags	LaunchDarkly / Azure App Config	A/B test model versions, prompt strategies, features
User Entitlement	GitHub subscription DB	Verify Copilot license (Individual, Business, Enterprise)

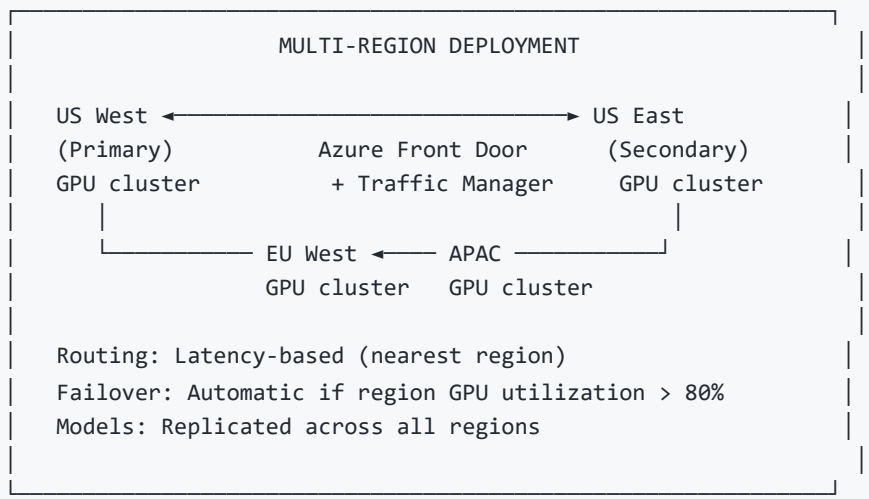
Scalability Considerations

1. GPU Cost Optimization

```
Problem: GPUs are the #1 cost driver (~$2-3 per GPU-hour for A100)

Solutions:
├── Request cancellation: 70-80% of requests never reach inference
├── Model tiering: Use small models for simple completions
├── Continuous batching: Pack multiple requests per GPU cycle
├── KV cache sharing: Common system prompts cached across requests
├── Quantization: INT8/INT4 for fast-tier models (2-4x throughput)
├── Speculative decoding: Small draft model + large verify (2x faster)
└── Auto-scaling: Scale GPU nodes based on queue depth
```

2. Global Distribution



3. Rate Limiting Strategy

```
Tier 1: Per-user token bucket
  ├── Individual: 100 requests/min
  ├── Business: 150 requests/min
  └── Enterprise: Custom limits

Tier 2: Per-organization aggregate
  └── Prevent one org from starving others

Tier 3: Global circuit breaker
  └── Shed load during GPU saturation (graceful degradation)

Graceful degradation order:
  1. Reduce max_tokens (shorter suggestions)
  2. Route to faster/smaller model
```

```
  3. Increase debounce delay on client
  4. Return empty suggestions (last resort)
```

4. Caching

```
Layer 1: Client-side (IDE extension)
  └── Cache recent completions, re-show if cursor returns to same position

Layer 2: Edge cache (API Gateway)
  └── Exact prompt match → return cached result (rare but free)

Layer 3: Semantic cache
  └── Similar prompts (cosine similarity > 0.95) → return cached result
  └── Reduces GPU load by ~5-10%

Layer 4: KV cache (GPU memory)
  └── Reuse computed attention for common prompt prefixes
  └── System prompt + common imports = pre-computed
```

Trade-offs

Decision	Option A	Option B	Copilot's Choice
Model hosting	Self-hosted GPUs	OpenAI API	**Hybrid** — use OpenAI for large models, self-hosted for fast models
Suggestion trigger	Every keystroke	After typing pause (debounce)	**Debounce** — dramatically reduces cost and noise
Context strategy	Current file only	Multi-file + RAG	**Multi-file + RAG** — better suggestions, higher accept rate
Streaming	Wait for full response	Stream tokens as generated	**Stream for chat**, wait for inline (ghost text needs full line)
Model selection	One model for all	Route by task complexity	**Route by task** — fast model for completions, large for chat/agents
Privacy	Send all code to cloud	Process locally	**Cloud with privacy controls** — users can exclude files, orgs control data retention
License filter	Always filter	Optional	**Optional per-org** — Enterprise can enable/disable duplicate

			detection
Offline support	Full offline mode	Cloud-only	**Cloud-only** with graceful degradation — local models emerging

---

Technologies Deep Dive

Model Serving Stack

Technology	Purpose	Why
vLLM	Inference engine	PagedAttention for efficient KV cache, continuous batching
TensorRT-LLM	NVIDIA optimization	Kernel fusion, quantization, 2-4x throughput vs vanilla
Triton Inference Server	Model serving	Multi-model management, metrics, health checks
CUDA	GPU compute	Direct GPU programming for custom kernels
ONNX Runtime	Cross-platform inference	CPU fallback, edge deployment

Infrastructure

Technology	Purpose	Why
Azure Kubernetes Service	Container orchestration	GPU node pools, auto-scaling, spot instances
Azure Front Door	Global load balancing	Latency-based routing, DDoS protection
Kafka / Event Hubs	Event streaming	Telemetry pipeline, async processing
Redis	Caching & rate limiting	Session state, token buckets, semantic cache
Cosmos DB	Chat history	Global distribution, low-latency reads

Azure Blob Storage	Code index	License filter bloom filters, RAG embeddings

Client-Side

Technology	Purpose	Why
VS Code Extension API	IDE integration	InlineCompletionProvider, Chat API
Language Server Protocol	Code intelligence	Symbol resolution, diagnostics, type info
Tree-sitter	Fast parsing	Extract syntax tree for context collection
AbortController	Request cancellation	Cancel in-flight requests on new keystrokes

---

How to Explain It Well

30-Second Pitch

> "Copilot is an AI pair programmer that provides real-time code suggestions. The IDE extension collects context — the code before and after the cursor plus relevant open files — and sends it to a GPU inference cluster using Fill-in-the-Middle encoding. We use a multi-model strategy: fast small models for inline completions under 300ms, and larger models for chat and agent tasks. The key challenges are latency optimization through debounce + cancellation, GPU cost management through request filtering and batching, and safety through license detection and secret scanning."

Key Talking Points

1. **Context is king**: The quality of suggestions depends on how well we assemble context (FIM, neighbor files, RAG)
2. **Cancellation saves millions**: 70-80% of requests are cancelled before inference — this is the biggest cost lever
3. **Multi-model routing**: Not every request needs GPT-4; route simple completions to fast small models
4. **Acceptance rate**: The north-star metric — measures both model quality and context quality
5. **Safety pipeline**: License filter + secret detection + responsible AI checks on every suggestion

Whiteboard Sequence

1. Start with IDE → API Gateway → Inference → Response flow
2. Zoom into context collection (FIM encoding, prefix/suffix)
3. Show the GPU cluster with batching and KV cache
4. Add post-processing (license filter, secret detection)
5. Overlay telemetry feedback loop (acceptance rate drives model improvements)

Common Follow-up Questions

"How do you handle latency for inline suggestions?"

- **Debounce** (150ms) avoids sending requests on every keystroke
- **Cancellation** aborts in-flight requests when user keeps typing
- **Fast models** (1-7B params) for inline, quantized to INT8
- **Speculative decoding** uses a tiny draft model verified by the main model
- **Edge routing** to nearest GPU cluster (< 30ms network RTT)

"How do you prevent suggesting copyrighted code?"

- Normalize and fingerprint model output using n-gram hashing
- Compare against a Bloom filter index of public code (~hundreds of millions of files)
- If match found, check license: permissive (allow + cite) vs copyleft (block)
- Enterprise customers can enable/disable this filter
- Also block verbatim matches > N lines regardless of license

"How do you manage GPU costs at scale?"

- **Cancellation** is the biggest lever (70-80% savings)
- **Model tiering**: Cheap small models handle most inline requests
- **Continuous batching**: Pack multiple requests per GPU forward pass
- **Quantization**: INT8/INT4 models use less VRAM, higher throughput
- **Spot/preemptible instances** for batch workloads (fine-tuning, indexing)
- **Auto-scale** GPU node pools based on queue depth, not CPU

"How does the agent mode work (multi-file edits)?"

- Agent receives a high-level task ("add authentication to this app")
- Uses a **ReAct loop**: Reason → Act (call tools) → Observe → Repeat

- Tools include: file read/write, terminal commands, search, web browse
- Each iteration sends updated context to a large model (GPT-4 class)
- User approves/rejects tool calls for safety
- Streams progress via SSE; final result = set of file diffs

"How do you handle multi-tenant isolation?"

- **Network isolation**: Each request authenticated via GitHub token → org membership verified
- **Data isolation**: No cross-tenant data leakage; code context only from user's own repos
- **Model isolation**: Shared model weights (no per-tenant fine-tuning on shared infra)
- **Telemetry isolation**: Org admins see only their org's usage metrics
- **Custom policies**: Orgs can block file patterns, set content exclusions, control data retention

"How would you add support for a new IDE?"

- Implement the IDE's extension/plugin API (IntelliJ Plugin SDK, Neovim Lua API, etc.)
- Reuse the same backend APIs (completions, chat, agents)
- Adapt context collection to each IDE's document model
- IDE-specific: ghost text rendering, inline diff UI, chat panel
- Use Language Server Protocol (LSP) for cross-IDE code intelligence